VerifyW3 — Benchmark Ecosistema Stellar

Investigación de Smart Contracts de Credenciales, Verificación y Atestaciones en Soroban

Octubre 2026

Propósito técnico: Identificar y analizar proyectos, estándares y contratos inteligentes en la red **Stellar / Soroban** con casos de uso equivalentes al de **VerifyW3** (registro descentralizado de hashes SHA-256 de certificados, verificación pública y revocación criptográfica). La consulta fue realizada directamente sobre el catálogo oficial del *Stellar Community Fund (SCF)* y el ecosistema de código de Soroban.

1. Proyectos Insignia del Ecosistema (Financiados por el SCF)

ACTA (did:stellar & Verifiable Credentials)

SCF Round #42 • $75,000 USD

Plataforma para emitir, verificar y revocar credenciales W3C on-chain vía Soroban. Implementa el estándar did:stellar y un contrato modular de listas de estado (*Status Lists*) para revocación inmediata.

Repositorios: `github.com/ACTA-Team/contracts-acta | github.com/ACTA-Team/contracts-acta-spikes`

Attest Protocol (Stellar Attestation Service)

SCF Round #32 • $83,050 USD

Infraestructura general de atestaciones on-chain y notarización en Soroban (análogo a Ethereum Attestation Service - EAS). Maneja registro e indexación mediante identificadores hash BytesN<32>, expiración y esquemas auditables.

Repositorios: `github.com/daccred/attestprotocol (Soroban SDK 27.0.6, Score 79/100)`

Chaincerts

SCF Round #18 • $103,125 USD

Solución pionera en el ecosistema Stellar para credenciales y certificados educativos verificables respaldados por contratos inteligentes en Soroban, desarrollada por la firma kommitters.

Repositorios: `github.com/kommitters/chaincerts-smart-contracts | github.com/kommitters/soroban-did-contract`

2. Repositorios de Referencia Directa (Casos de Uso Idénticos)

Proyecto / Repositorio	Aporte Arquitectónico	Firmas Clave en Rust (Soroban SDK)
Hackathon-Stellaroid_Earn `Iron-Mark/Hackathon-Stellaroid_Earn`	El más completo: Registro de emisores, emisión con BytesN<32> para el hash del certificado, verificación pública y revocación. Conectado con stellar-wallets-kit.	• register_issuer(issuer, ...) • register_certificate(..., cert_hash: BytesN<32>, ...) • verify_certificate(verifier, cert_hash) • revoke_certificate(actor, cert_hash)
Digital-certificate-validator-full `sohansarkar07/Digital-certificate...`	Separa el registro institucional del validador. Soporta flags de estado (revocado, suspendido, activo) y conteo global.	• issue_typed_credential(cert_hash, ...) • verify_certificate(cert_hash) -> bool • revoke_certificate(cert_hash) • is_revoked(cert_hash) -> bool
Digital-Certificate-Validator (Basic) `late-cat/Digital-Certificate-Validator`	Contrato minimalista y limpio. Excelente referencia rápida para estudiar el ciclo de vida básico en Soroban.	• issue_certificate(cert_id, course, ...) • revoke_certificate(cert_id) • is_certificate_valid(cert_id) -> bool
soroban-certificate-onchain `GamaNaufal/soroban-certificate...`	Manejo de emisores con lista blanca (add_issuer / is_registered_issuer) y consultas de portafolio.	• add_issuer(issuer: Address) • issue_certificate(issuer, ...) • revoke_certificate(issuer, cert_id) • verify_certificate(cert_id)

3. Buenas Prácticas y Criterios Adoptados para VerifyW3

1. Indexación Directa por Hash (BytesN<32> o String): Indexar el certificado por su hash SHA-256 en DataKey::Record(hash) asegura que cualquier verificación pública en el frontend tenga un costo computacional y tiempo de consulta **O(1)**.

2. Privacidad por Diseño (Ventaja Competitiva): A diferencia de repositorios que almacenan nombres y notas en texto plano en la cadena, VerifyW3 guarda exclusivamente metadatos mínimos (hash, cid, title, issuer, timestamp y revoked). Los datos personales sensibles se mantienen fuera de cadena (Off-Chain), cumpliendo normativas de protección de datos personales (Habeas Data / GDPR) y minimizando el uso de storage.

3. Autorización Criptográfica Estricta (issuer.require_auth()): Toda emisión y revocación debe requerir la firma digital de la billetera de la institución emisora, impidiendo alteraciones de terceros no autorizados.

4. Persistencia y Gestión de TTL: Se utiliza env.storage().persistent() con extend_ttl (patrón oficial de la Sesión 5 y 7 de Stellar), garantizando que los registros no expiren ni sean eliminados del ledger por inactividad.